

PRAHARI — an early-warning system for India's oil supply

PRAHARI watches the world for trouble that could choke off India's oil supply. The moment something looks dangerous, it does the work a room full of analysts would normally take weeks to do: it figures out how much the trouble costs India (in rupees, and in days of supply left), and it tells the people who buy crude what to buy instead and how to ship it. "Prahari" means watchman.

Built for Challenge 2 (supply-chain intelligence and energy security).

Our one-line pitch: **47 days versus 47 seconds.**

Why we picked this problem

India buys about 88% of its crude oil from other countries. Close to 40% of that oil passes through one narrow gap of water next to Iran, the Strait of Hormuz. If that strait becomes dangerous, a huge share of the country's oil is suddenly in doubt.

The trouble is speed. When a crisis hits, prices jump, refiners start scrambling, and it takes weeks to line up other supply. McKinsey measured it: a country that isn't prepared loses roughly 47 extra days getting back to normal after an oil shock. India's emergency reserve only holds about 9 and a half days of cover.

Here is the gap we went after, in plain terms. The danger shows up in the news and in ship movements almost the instant it happens. The response takes weeks. We wanted to shrink that from weeks to seconds.

What it actually does

Think of an ordinary day. Oil prices tick along, ships move through Hormuz, nothing unusual, and our dashboard shows a calm map with a low risk score.

Now a threat appears. Say news breaks of a strike near the strait. Everything below runs in a single pass:

1. **It reads the risk.** It gathers the recent news events for that sea corridor, weighs each one by how serious and how recent it is, and boils that down to a single risk number between 0 and 1.
2. **It simulates the damage.** It runs 10,000 fast simulations of where the oil price could head, then follows that through to what it means for India: how much bigger the daily import bill gets, and how quickly the emergency reserve drains.
3. **It finds the way out.** Using a map of who supplies India's crude and the routes they use, it works out which suppliers can still deliver while steering clear of the dangerous chokepoints, ranks them by speed and safety, and draws the new route on screen.
4. **It writes the answer in one line.** Something a decision-maker reads in five seconds: the risk level, the cost per day, the reserve days left, and the supplier and route to switch to.

From the signal arriving to the recommendation appearing, this runs in about 20 milliseconds on our laptop. The "47 seconds" is the promise on the slide; the real number is much faster than that.

A real run from our data: the Strait of Hormuz reads Critical at 0.90, backed by 77 actual news events, with an estimated cost of roughly Rs 1,300 crore a day, and the recommended fix is to reroute through the UAE's Fujairah port, about 3 days out. Nobody typed that answer in. It falls out of the data.

The idea that makes it fast

Most price-shock models have a hard job built in: they have to guess *when* a shock will strike, because they only ever see it after prices have already moved. We sidestep that. We watch the news and the ships directly, so we catch the shock while it is forming. We never have to guess the timing. We only have to size the shock and estimate how long it lasts. That is a smaller and more honest job, and it is the reason the whole system can answer in near real time.

What's under the hood

Six data feeds. Oil prices from the US government (EIA) and from Yahoo Finance, the US sanctions list (OFAC), live ship positions (AIS), India's monthly import figures (PPAC), and world news events (GDELT). Small scripts pull each one and drop it into one shared database.

One database. Everything lands in a single cloud Postgres database, so the map, the model, and the agents all read the exact same numbers. No two parts of the system can disagree about what the data says.

A knowledge graph. A small map of India's real crude supply chain: 7 suppliers, their export ports, 5 sea chokepoints, Indian ports, and 5 refineries, wired together the way the real infrastructure is (including the pipelines that bypass Hormuz, like the UAE's line to Fujairah). The map itself is fixed, but the danger on each chokepoint updates from the live news. Ask it "who can still reach Jamnagar today" and it answers with real routes and travel times.

A scenario model. A price-shock simulation (jump-diffusion, 10,000 runs) that turns a risk score into a spread of outcomes: the likely price jump, the worst-case jump, how far the reserve falls, and the cost per day. It reports a range, not a single fake-precise number.

An agent pipeline. The four steps above are wired into one chain using LangGraph, so a detected signal actually drives the whole thing end to end and we can show each step as it runs.

A dashboard. A single-screen React and Leaflet console: a live map, a risk dial, the chokepoint watch list, the simulation chart, the cost in rupees and days, and the ranked list of suppliers to switch to. There is also a panel of assumptions you can change on the spot and watch everything recompute.

The one rule we stuck to: the maths is code, only the words come from AI

Every number in the system is computed and can be checked by hand: the risk score, the simulation, the travel times, the cost. The only place we use a language model is to read messy news text and turn it into clean events, and (in one version) to write the summary sentence. No figure is ever invented by an AI. That keeps the whole thing auditable, which matters when you are telling a government what a crisis costs.

Is the data real?

Yes. All of it is real, public data. Real news, the real US sanctions list, real oil prices, real Indian import figures, real ship positions. We made none of it up.

We should be straight about one thing, though: it is not a live stream. Each feed refreshes on its own clock. Prices and ship positions update every ten minutes or so, news every few hours, and the slower government figures daily or monthly. For the demo we run on a recent real snapshot and refresh it just before we present. One honest limit: the free ship-tracking service barely covers the Persian Gulf, so the live ship dots you see sit on well-covered lanes like the English Channel, there to prove the pipeline really is live. The Hormuz story is told with the data we do have, which is the geography, the sanctioned tankers, and the news signal.

Our rule the whole way through: real but slightly old beats made-up. We never invent a number to fill a gap.

What we kept simple on purpose

Time was short, so we cut things that would have added risk on stage without changing the story. The final summary sentence comes from a template rather than an AI, so nothing can hang at the worst possible moment. We run one price model instead of two. We left out the heavier real-time streaming setup and a couple of other extras. Every one of those was a choice, not an oversight, and we are happy to walk a judge through the reasoning on any of them.

How to run it

See [DEMO_GUIDE.md](#) for the step-by-step. The very short version:

```
./run_dev.sh
```

then open <http://localhost:5173> and click **Inject signal**. That button runs the whole pipeline on a cached real event with no database and no internet, so it works even if the wifi dies.

The team

Team **f20230436**

- Ansh Sharma (team lead)
- Harsh Vardhan
- Aditya Nitin Jagtap
- Abhijay Pansari

The four of us split the build across four areas: the data layer (six live feeds into one shared database), the news pipeline that turns raw world events into scored risk signals, the agents and the knowledge graph, and the backend API and the dashboard.